

복잡한 운영 데이터 관리 시스템과 웹 성능을 직접 설계하는 프론트엔드 개발자



정다훈 웹 프론트엔드 개발자

휴대폰 010-4346-0429

Email dahun428@naver.com

주소 경기 구리시 인창동

Portfolio github.com/dahun428-fx

요약

6년차 프론트엔드 개발자. React·TypeScript·Next.js·TanStack Query 기반으로, 역할별 데이터 조회·대용량 대시보드·관리자 도구 같은 복잡한 운영 데이터를 다루는 백오피스·운영 시스템과 그 성능을 직접 설계해 왔습니다.

삼성물산 데이터 플랫폼(내담씨앤씨 SI)에서 대용량 테이블·시계열 차트 렌더링 시간을 **2.5초에서 1초대**로 단축하고 역할별 조회·필터·정렬 구조를 Spring REST API부터 화면까지 직접 설계했으며, 글로벌 B2B 커머스 레거시 전환부터 임직원 건강 플랫폼 MySQL 모델·Spring Boot·Web·Admin End-to-End 구축까지 풀스택 범위로 운영팀이 쓰는 관리자 도구를 반복해 만들어 온 개발자입니다.

기획·AI·백엔드·운영 조직과 API 응답 정책을 직접 조율하며 WBS·ETA 기반으로 다중 우선순위 업무를 주도적으로 진행해 왔습니다.

핵심 성과

③ 대용량 데이터 렌더링·쿼리 성능 설계

구간 분리 측정·FE 렌더링과 BE 인덱싱 양단 개입·병목 해소·역할별 데이터 조회 설계

→ 수만 건 검색 응답 5초 → 1초, 대시보드 렌더링 2.5초 → 1초대

② 대규모 레거시 전환

기술 리드·글로벌 B2B 커머스·PHP·jQuery → Next.js·TypeScript·FE-BE End-to-End

→ 6인 팀 리드로 전면 전환 완수, 평균 로딩 약 50% 개선·페이지 접근 8초 → 2초 / 대용 플랫폼 108개 페이지 9주 단독 내재화

① AI 건강검진 챗봇 제품화

SaaS 구조 완성·FE AX 전환·분석-설계-구현 전 과정 주도

→ 임직원 3,493명 실사용, 신규 고객사 FE 커스터마이징 2주 납품 — 최초 구현 10주 대비

④ 소통과 기술 리더십

조직의 일하는 방식 전환·기획-운영 커뮤니케이터·표준화-문서화-조직 자산화

→ Vitest·Playwright E2E 하네스 단독 구축, 팀 10명 전원의 개발 절차로 정착, 세미나 12회·기술 문서 48건

대웅제약 AI추진팀

2025.07 ~ 재직중

풀스택 Product Engineer / 비즈36.5·AI코치·바이오에이지 신규 개발 및 유지보수 / 팀 AX 전환 리드

- AI추진팀 **다나아데이터 스쿼드** (부서 20명 · 스쿼드 10명)
- 비즈36.5·AI코치·바이오에이지 **3개 서비스 FE 총괄**
- **Web·Admin** 관리자 도구 및 데이터 조회·처리 **인터페이스 End-to-End 개발** (관리자/운영 시스템)
- **신규 기능 End-to-End 개발** (FE React·Next.js / BE Node.js·Spring Boot·Nest.js·MySQL·PostgreSQL·MongoDB)
- **LLM 서버 개발 협업** — 프롬프트 구성·응답 후처리 개선과 VectorDB·RAG 구축 참여 (Python·LangChain)
- CI/CD·테스트 자동화, 운영 서버 **Blue-Green 무중단 배포** (Jenkins·Docker·Playwright)
- **팀 AX 전환 리드** (Claude Code·Codex·Harness Engineering)

내담씨앤씨 SI사업부

2020.10 ~ 2025.06

웹 풀스택 개발자 / 삼성물산·한국미스미 프로젝트

- 한국미스미 6명 · 삼성물산 10명 규모 프로젝트 팀에서 개발
- 한국미스미 **Global B2B 커머스** 신규 개발과 레거시 전환, **일본 본사와 일본어 직접 협업** (PHP·jQuery → Next.js·TypeScript)
- 삼성물산 사내 웹·앱 플랫폼 신규 개발·유지보수 (Vue 3·React Native·Spring·Oracle)
- **대용량 데이터 처리 및 처리 소요 시간 단축** (대시보드 렌더링 2.5초 → 1초 · 수만 건 검색 응답 5초 → 1초)

1. 대응제약 | AI 건강검진 챗봇 제품화

2025.07 ~ 2025.10

[주요 업무]	AI 챗봇 신규 개발에서 프론트엔드 총괄 — 아키텍처 분석·설계부터 구현·테스트까지 수행, 응답 정책·AI 서버 로직 등 제품 전반의 기술 결정에 관여
[문제]	AI 건강검진 챗봇을 10주 내에 실서비스로 올려야 했습니다. MVP 정의와 PoC 검증을 거친 뒤 프론트엔드 전체를 구현해야 했고, LLM 출력은 형식과 품질이 매번 달라 화면에서 무엇을 보장할 수 있는지부터 정의해야 했습니다.
[주요 실행]	- 아키텍처·정책 설계 기획자와 MVP 범위를 정하고, AI 개발자와 응답 형식·스트리밍 방식·오류 정책을 먼저 합의해 제품 아키텍처의 전제 확정 - AI 채팅 경험 구현 SSE 기반 실시간 스트리밍과 중지·재시도·재생성·상답 이력·답변 피드백 흐름 구현, Markdown 응답을 표·차트·링크 React 컴포넌트로 렌더링하는 계층 설계, XSS-404/500·LLM 오류를 4유형으로 분류해 복구 전략 적용 - 도메인 화면 구현 건강검진 결과·건강 점수·만성질환·이상 결과·종합 소견 등 12개 도메인 모듈과 브라우저 언어 감지 기반 다국어 환경 구축 - SaaS 확장 구조 전환 기능별 차등 과금 요구를 아키텍처로 번역, 이미 완성된 코드베이스의 대규모 리팩토링을 감수하고 Base-Theme·Config-Driven UI로 전환해 운영·기획자가 고객사별 테마·기능 노출을 직접 제어하도록 구성 - LLM 서버 협업 Python 프롬프트 구성·응답 후처리 개선과 VectorDB·RAG 구축 참여
[성과]	- 실사용 서비스 전환 임직원 3,493명 규모 서비스로 계획 대비 2주 조기 전환 - SaaS 제품화·신규 고객사 납품 테넌트별 콘텐츠·테마·기능 플래그를 제어하는 관리자 프리뷰 도구 개발로 Config-Driven SaaS 구조 완성, 신규 고객사 FE 커스터마이징 2주 납품 — 최초 구현 10주 대비 - FE 완성도 실사용 발화 400여 건 검증 전체 통과, SSE·React Query 캐싱 설계로 AI 응답 평균 2초대·Lighthouse 91점 - 품질 체계 Vitest·Playwright E2E로 반복 QA 3시간 → 30분, PostHog·Web Vitals 기반 사용성 4.18점·만족도 4.06점 확보

기술: React · TypeScript · TanStack Query · Recoil · SSE · Vitest · Storybook · Playwright · PostHog · Node.js · MongoDB · Nginx · Python · FastAPI · LLM

2. 대응제약 | 임직원 건강검진 통합플랫폼 구축

2025.11 ~ 2026.02

[주요 업무]	2019년 레거시 검진 서비스를 React 기반 임직원 건강검진 통합플랫폼으로 현대화 — 서비스·DB 구조 분석부터 점진 전환 설계, MySQL·Spring Boot·REST API·화면 End-to-End 개발, 배포 자동화까지 풀스택 총괄
[문제]	외주 중심으로 운영되던 검진 예약 서비스를 내부 개발·운영 가능한 임직원 건강 플랫폼으로 전환해야 했습니다. jQuery·Thymeleaf 화면, Spring Boot 서버 로직, MySQL 데이터 구조가 기능별로 결합돼 변경 영향 범위 파악이 어려웠고, 사용자 권한·메뉴·데이터 출력 정책이 Web과 Admin 간에 일관되지 않아 운영팀의 관리 부담이 컸습니다.
[주요 실행]	- 서비스·데이터 구조 분석 기존 소스와 DB 스키마를 분석해 화면 요청부터 Controller·Service·Query·DB·출력까지 전 구간 정리 - 점진 전환 구조 설계 전면 재작성의 일정·회귀 리스크와 jQuery 장기 공존의 유지비·DOM 충돌을 모두 저울질한 끝에 React를 화면 단위로 공존·재사용하는 점진 전환을 직접 설계, 변경·장애 빈도를 기준으로 전환 순서 결정 - 풀스택 End-to-End 개발 신규 건강관리 기능에 필요한 MySQL 데이터 모델·Spring Boot 서버 로직·REST API·Web·Admin 화면 End-to-End 개발 — 사용자 권한·메뉴·브랜딩·데이터 출력 정책을 공통 기준으로 정리해 고객사별 배포 대응 구조 확보 (관리자 도구·데이터 조회·처리 인터페이스) - BFF 계층 신규 구축 React 전용 데이터 집계·프록시 API를 Node.js로 구축, 모놀리식 BE-FE 분리 전략의 일환으로 RESTful 구조 점진 도입 - AI 기반 회귀 검증 82개 화면 전환의 UI 회귀 검증을 Playwright 기반 AI E2E 테스트로 자동화해 반복 UI 검증을 사람 대신 수행 - 배포 자동화 FileZilla 수동 이관에 의존하던 배포를 개발 서버 Jenkins 파이프라인과 운영 서버 Docker 기반 Blue-Green 무중단 배포로 전환, WebView 호환성 검증 기준 수립 (크로스 브라우저·반응형 웹)
[성과]	- 통합플랫폼 도약 2019년 레거시를 React 기반 임직원 건강검진 통합플랫폼으로 재구축하고 UI/UX 전면 개편을 주도 - 단독 풀스택 전환 108개 페이지·82개 화면을 9주 내 단독 전환, 외주 없이는 못 고치던 서비스를 내부 개발·운영 구조로 전환 - 데이터-투-화면 E2E 오너십 신규 건강관리 기능을 MySQL 모델부터 서버·API·화면까지 End-to-End 단독 개발, 고객사별 CI·메뉴·기능 노출 변경을 1주 내 대응 가능한 운영 구조 확보 - 검증·배포 자동화 수동 FTP 배포를 Jenkins·Docker 파이프라인으로 전환해 배포 리드타임 10분 → 2분(약 80%) 단축, 운영 서버는 Blue-Green 방식으로 서비스 중단 없는 릴리스 확보

기술: React · TypeScript · Java · Spring Boot · REST API · MySQL · Node.js · Express · jQuery · Thymeleaf · Playwright · Jenkins · Docker · WebView

3. 대응계약 | 팀 AX 전환 리드 — 하네스 엔지니어링과 개발 표준화

2026년 상반기

[주요 업무]

팀의 AI 활용 개발 방식 전반을 설계·리드 — AI 생성 코드 검증을 위한 하네스 엔지니어링 구축, FE AX 개발 표준 단독 설계, 품질·운영 자동화와 팀 지식 자산화 주도

[문제]

AI 코딩 에이전트를 도입하면 병목은 코드를 만드는 속도가 아니라 만들어진 코드를 무엇으로 믿을지로 옮겨갑니다. 도입은 됐지만 팀원마다 활용 방식이 달랐고, AI가 만든 코드를 누가 어떤 기준으로 신뢰할 것인지가 정의돼 있지 않았습니다. 그 위에 수동 발화 테스트·반복 UI 개발·배포 전후 확인·운영 지표 조회의 병목이 겹쳐 있었습니다.

[주요 실행]

- 하네스 엔지니어링 구축
발화 테스트·LLM 응답 검증·타입/빌드·배포 전후 확인을 **자동 실행되는 검증 하네스로 통합**하고 Playwright E2E를 **배포 파이프라인의 필수 게이트로 편입** — 사람이 판정하지 않아도 같은 기준으로 품질이 걸러지는 구조
- FE AX 개발 표준 단독 설계
AI 생성 코드를 팀 전체가 같은 기준으로 검증·신뢰할 수 있도록 컨텍스트 관리·금지 패턴·보안 위험·검증 절차를 **FE AX 개발 표준으로 설계, 사내 공식 문서로 채택**
- 공통 컴포넌트 표준화
Storybook 스토리 210개로 재사용 컴포넌트를 카탈로그화하고 기획자와 UI를 조기 확정해, 신규 화면 조립을 공통 컴포넌트 기반으로 전환
- 운영 확인 자동화
GA4·PostHog 기반 운영 데이터 대시보드 구축, 배포 전후 상태와 운영 지표를 비개발 직군도 직접 확인 가능하게 전환
- 학습의 조직 이식
외부 컨퍼런스·전문가 인사이트를 흡수해 **개발 세미나 12회**로 팀에 이식하고, 검증 기준·노하우를 **기술 문서 48건으로 자산화**해 신규 합류자도 같은 기준으로 개발 가능하게 정비

[성과]

- 일하는 방식의 전환
10명 팀에서 단독 설계한 개발 표준과 검증 하네스가 **전원의 개발 방식으로 정착**, E2E 필수 게이트로 담당자 부재 시에도 같은 기준의 검증·배포 유지
- 인력 효율 향상
반복 UI 검증을 자동 판정으로 대체하고 반복 컴포넌트 개발을 **90분 → 15분(약 83%)**으로 단축, 검증 인력을 개발에 재투입
- 협업 리드타임 단축
Storybook 기반 UI 조기 확정으로 기획-개발 검증 **리드타임 2일 → 1일**
- 유지보수성 향상
공통 컴포넌트 카탈로그와 개발 표준·기술 문서 48건으로 코드 일관성·인수인계 기반 확보, 운영 데이터 확인 3단계 → 1단계 자동화

기술: Codex · Claude Code · Playwright · Storybook · Jest · ESLint · TypeScript · GA4 · PostHog · Jenkins · GitLab CI/CD

4. 삼성물산 | 대용량 데이터 플랫폼·모바일 서비스 고도화

2024.10 ~ 2025.04 · 내담씨앤씨 SI

[주요 업무]	현장 데이터 플랫폼의 Web 대시보드(Vue 3)와 현장 작업자용 모바일 앱(React Native)의 성능 개선·기능 고도화 — 병목 진단부터 FE 렌더링 설계, BE 인덱싱·응답 경량화까지 전 구간 수행
[문제]	Web 대시보드는 대용량 테이블·시계열 데이터를 빠르게 제공해야 했고, 모바일 앱은 반복 API 호출과 플랫폼별 동작 차이로 사용자 대기 시간과 운영 부담이 발생했습니다.
[주요 실행]	- 구간 분리 측정 진단 "API가 느릴 것"이라 추정하는 대신 API 응답 시점과 화면 노출 시점을 분리 측정해 병목을 클라이언트 렌더링 구간으로 특정 - 대시보드 렌더링 설계 Vue 3·ECharts·RealGrid 대용량 화면의 렌더링 생명주기 분리와 Lazy Rendering 적용 — 대용량 테이블·시계열 차트 초기 처리량 최적화 - 클라/서버 경계 설계 데이터 규모를 기준으로 클라이언트 필터링과 서버 검색의 경계를 정의, Spring REST API와 역할별 데이터 조회·필터·정렬 로직을 함께 설계해 사용자 권한별 데이터 표현 구조 구성 - 모바일 목록 최적화 효과가 가장 컸던 FlatList 가상화·renderItem 재렌더링 제거를 중심으로 안정적 key 설계·getItemLayout 적용, debounce·캐싱·중복 요청 취소는 보조 개입으로 병행 — 검색 때마다 API를 호출하던 구조를 화면 진입 시 비동기 선조치 후 Recoil에 저장하는 방식으로 변경 - 백엔드 응답 경량화 DB 인덱싱과 DTO 투영으로 응답 경량화 직접 수행
[성과]	- 대시보드 성능 개선 FE 렌더링 생명주기 분리와 BE 인덱싱·DTO 투영을 양단에서 단독 적용해 대용량 데이터 렌더링 2.5초 → 1초대 단축 - 모바일 검색 개선 수만 건 목록의 렌더링 병목을 제거해 검색 결과 노출 5초 → 1초(약 80%) 단축

기술: Vue 3 · React Native · TypeScript · ECharts · RealGrid · Java · Spring · REST API · MyBatis · Oracle · TanStack Query

5. 한국미스미 | 글로벌 B2B 커머스 Next.js 전면 전환

2022.06 ~ 2024.10 · 내담씨앤씨 SI · 3개 프로젝트

- [주요 업무]** 일본 본사 주도의 전 지사 아키텍처 통일 프로젝트에서 한국 팀 React 전문 개발자로서 6인 팀의 마이그레이션과 기술 의사결정을 주도 — 사용자 화면 개발에서 시작해 아키텍처·성능·다국어·배포·모니터링까지 책임 범위를 확장
- [문제]** PHP·jQuery 기반 글로벌 B2B 쇼핑몰은 화면 간 결합도가 높아 기능 확장과 유지보수가 어려웠고, 긴 초기 접근 시간과 단일 렌더링 방식으로 성능과 검색 노출을 함께 최적화하기 어려웠습니다. 일본 본사의 전 지사 아키텍처 통일 결정을 계기로 한국 웹의 전면 전환을 이끌어야 하는 상황이었습니다.
- [주요 실행]**
- **한국 웹 전환 리드**
한국 팀에서 React를 가장 깊이 다루는 개발자로서 **6인 팀의 마이그레이션을 이끌고**, 일본 본사 개발자들과 **일본어로 직접 소통하는 크로스보더 협업**으로 전면 전환을 완료까지 견인
 - **렌더링 경계 판단**
상품·검색 유입 페이지는 SEO와 초기 로드를 위해 SSR로, 로그인 후 상호작용 중심 페이지는 서버 부하·번들 비용을 고려해 CSR로 분리 — **페이지별 렌더링 비용을 기준으로 경계를 직접 판단**
 - **사용자 기능 구조화**
상품 비교·멀티다운로드 등 복잡한 기능을 **공통 컴포넌트와 Custom Hook으로 구조화**하고, Redux·브라우저 저장소로 사용자 상태 일관성 확보
 - **성능 최적화 직접 적용**
Lighthouse 기반 병목 분석 후 라우트·컴포넌트 단위 code splitting·next/image 최적화·next/link prefetch·번들 청크 분할·Lazy Loading 적용
 - **글로벌 요구 대응·관측성 운영**
i18next 기반 **영·중·일 다국어 구조와 SEO 대응**, Lighthouse·GA·Adobe Analytics·Datadog으로 **성능·SEO·오류 지표화**, Vercel 배포 자동화
- [성과]**
- **전면 전환 완수**
한국 법인 웹을 PHP·jQuery에서 **Next.js·React·TypeScript로 전면 전환 완료**, 전 지사 아키텍처 통일의 한국 파트 완수 — 기존 PHP 서비스 대비 평균 로딩 속도 약 50% 개선
 - **성능 개선**
유지보수·성능 개선 프로젝트에서 Lighthouse 병목 분석과 리소스 최적화로 **페이지 접근 시간 8초 → 2초 단축**
 - **사용자 경험 개선**
인터랙션 기능 고도화 이후 **사용자 체류 시간 32% 증가**

기술: Next.js · React · TypeScript · JavaScript · Redux · RxJS · i18next · PHP · jQuery · Twig · Vercel · Datadog · Google Lighthouse · GA · Adobe Analytics

프론트엔드

React · Next.js · Vue 3 · React Native · TypeScript · JavaScript

상태관리 · 데이터 페칭

TanStack Query · Redux · Recoil · Zustand

UI · 시각화

Tailwind CSS · ECharts · RealGrid · i18next

백엔드

Node.js · Express · Nest.js · Java · Spring Boot · Python · FastAPI · REST API

데이터베이스

MySQL · Oracle · PostgreSQL · SQLite · MongoDB

품질 · 검증

Playwright · Vitest · Jest · Storybook · Chromatic · ESLint

배포 · 인프라

Jenkins · Docker · GitLab CI/CD · Vercel · AWS S3 · EC2 · Nginx

분석 · 관측

Datadog · GA4 · Adobe Analytics · PostHog · Lighthouse

협업

Figma · Jira · Slack · Confluence · Notion · Git · GitLab

AI 활용 개발

Claude Code · Codex · Harness Engineering · SSE · LangChain · RAG · VectorDB

학력

성공회대학교 일어일본학과 졸업

2010.03 ~ 2017.02

복수전공: 경영학과 · 학점: 3.8 / 4.5

여의도고등학교 졸업

2007.03 ~ 2010.02

해외경험

일본 워킹홀리데이 · 무인양품 도쿄 지사 근무

2018.09 ~ 2019.06 · 10개월

일본 현지 매장 실무를 비즈니스 일본어로 수행 — JLPT 1급 실전 활용

교육

웹 콘텐츠 개발을 위한 응용SW엔지니어링 · 중앙에이치티에이(주)

2020.03 ~ 2020.09

Java 기반 웹 애플리케이션 백엔드 · 프론트엔드 개발 과정 이수

자격증

정보처리기사 · 한국산업인력공단

2020.08

어학

영어 · TOEIC 825점

2024.05

일본어 · JLPT 1급

2018.08